

Group Map

<table><tr><th colspan="2">Table 2</th></tr><tr><td colspan="2">Irene Blanco Casalla William Nimtz</td></tr><tr><td>Sunyoung You</td><td>Sujal Vajire</td></tr></table>	Table 2		Irene Blanco Casalla William Nimtz		Sunyoung You	Sujal Vajire	<table><tr><th colspan="2">Table 3</th></tr><tr><td>Max Hortop</td><td>Lautaro Garcia</td></tr><tr><td>Dashiel Matlock</td><td>Lin Tian</td></tr></table>	Table 3		Max Hortop	Lautaro Garcia	Dashiel Matlock	Lin Tian	<table><tr><th colspan="2">Table 4</th></tr><tr><td>Jessica Duran</td><td>Ethan Kepros</td></tr><tr><td></td><td>Mohamed</td></tr><tr><td></td><td>Abdulazim Ali Afifi El</td></tr><tr><td>Aidana Bekbulatkyzy</td><td>Habet</td></tr></table>	Table 4		Jessica Duran	Ethan Kepros		Mohamed		Abdulazim Ali Afifi El	Aidana Bekbulatkyzy	Habet	<table><tr><th colspan="2">Table 5</th></tr><tr><td>Sam Hu</td><td>Jack Bajcz</td></tr><tr><td>Basma AlMahmood</td><td>Samuel Goodluck</td></tr></table>	Table 5		Sam Hu	Jack Bajcz	Basma AlMahmood	Samuel Goodluck
Table 2																															
Irene Blanco Casalla William Nimtz																															
Sunyoung You	Sujal Vajire																														
Table 3																															
Max Hortop	Lautaro Garcia																														
Dashiel Matlock	Lin Tian																														
Table 4																															
Jessica Duran	Ethan Kepros																														
	Mohamed																														
	Abdulazim Ali Afifi El																														
Aidana Bekbulatkyzy	Habet																														
Table 5																															
Sam Hu	Jack Bajcz																														
Basma AlMahmood	Samuel Goodluck																														
<table><tr><th colspan="2">Table 1</th></tr><tr><td>Abraham Rai</td><td>Alexander Lover</td></tr><tr><td>Xin Dong</td><td>Jingyi Ge</td></tr></table>	Table 1		Abraham Rai	Alexander Lover	Xin Dong	Jingyi Ge		<table><tr><th colspan="2">Table 7</th></tr><tr><td>Saksham Mamtani</td><td>Andrew Tran</td></tr><tr><td>Sanjida Meherin</td><td>Mi Zhou</td></tr></table>	Table 7		Saksham Mamtani	Andrew Tran	Sanjida Meherin	Mi Zhou	<table><tr><th colspan="2">Table 6</th></tr><tr><td>Madison Mercer</td><td>Md Shariful Islam</td></tr><tr><td>Grace Akintan</td><td>Michael Dittman</td></tr></table>	Table 6		Madison Mercer	Md Shariful Islam	Grace Akintan	Michael Dittman										
Table 1																															
Abraham Rai	Alexander Lover																														
Xin Dong	Jingyi Ge																														
Table 7																															
Saksham Mamtani	Andrew Tran																														
Sanjida Meherin	Mi Zhou																														
Table 6																															
Madison Mercer	Md Shariful Islam																														
Grace Akintan	Michael Dittman																														

CMSE 801 - Day 4

Practice with `if` statements and functions;
Revisiting the savings model

Jan 22, 2025



General Course Announcements

- Homework 1 should come out on Next Monday. This assignment will be **due at 11:59pm on Saturday, Feb 21.**
 - This will be an opportunity to showcase all of the Python fundamentals you've picked up in course up to this point.
- Reminder: office hours/help room details are on the course homepage:
<https://cmse.msu.edu/cmse801>
- Remember: at the end of class **you will be uploading your in-class notebook to D2L.**

Questions/comments from Day 4 PCA

- More practice with if statements? **In-class assignment will help**
- How to inspect variables like in MATLAB?
- General questions around writing functions from scratch and how they work
 - the hypotenus example -- **we'll look at that one**
- Setting default values
- Some questions and confusion about "variable scope"
 - Python is pretty lax about variable scope and jupyter notebooks can make this a bit more confusing since you can run code "out of order". This can make things even more confusing!
 - **We'll explore this a little bit as well**

Variable Inspector

Easier to search in the Jupyter Lab extension manager. I found a random one `lckr-jupyterlab-variableinspector` (3.2.4).

In [1]:

```
x = [1 , "something",4,"ok"]
```

You will get something look like this:

The screenshot displays the JupyterLab interface with the following components:

- PYPI MANAGER:** A sidebar on the left showing installed extensions. The 'jupyterlab-variableinspector' extension (version 3.2.4) is highlighted.
- Context Menu:** A menu is open over the code editor, listing actions such as 'Cut Cell', 'Copy Cell', 'Paste Cell Below', 'Delete Cell', 'Split Cell', 'Merge Selected Cells', 'Merge Cell Above', 'Merge Cell Below', 'Clear Outputs of All Cells', 'Enable Scrolling for Outputs', 'Disable Scrolling for Outputs', 'Undo Cell Operation', 'Redo Cell Operation', 'Restart Kernel...', 'New Console for Notebook', 'New Subshell Console for Notebook', 'Show Contextual Help', 'Show Log Console', 'Shift+Right Click for Browser Menu', and 'Open Variable Inspector'.
- Variable Inspector:** A panel on the right titled 'Variable Inspector' showing the state of the 'Python 3 (ipykernel)' environment. It displays a table with the following data:

NAME	TYPE	SIZE	SHAPE	CONTENT
x	list	88	4	[1, 'something', 4, 'ok']

The code editor shows the following code cell:

```
[1]: x = [1, "something", 4, "ok"]
```

The importance of knowing if your code is "inside" or "outside" your conditional statement

In [2]:

```
# initializing the list
greater_than_6_list = []
for i in range(11):
    print("i equals",i)
    if i > 6:
        print("i is greater than 6")
        # appending the number that is greater than 6
        greater_than_6_list.append(i) # I need to make sure this is "inside" the loop to do what I wan
print("List of numbers to sum:",greater_than_6_list)
print("Sum of numbers is equal to:",sum(greater_than_6_list))
```

```
i equals 0
i equals 1
i equals 2
i equals 3
i equals 4
i equals 5
i equals 6
i equals 7
i is greater than 6
i equals 8
i is greater than 6
i equals 9
i is greater than 6
i equals 10
i is greater than 6
List of numbers to sum: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Sum of numbers is equal to: 55
```

Demystifying function creation

What are the important components needed when creating a function?

LET'S LIST SOME IMPORTANT PARTS OF WRITING FUNCTIONS:

- `def` -- defines the function
- name (comes right after `def`)
- arguments -- inside `( )`
- `return` -- use this to get something back from the function
- It should serve a purpose

In [3]:

```
# Calculating the hypotenuse of a triangle  
a = 3  
b = 4  
hyp = (a**2 + b**2)**(1/2)  
print(hyp)
```

5.0

In [4]:

```
# Can we create a function that does this?  
def get_h(a,b):  
    hyp = (a**2 + b**2)**(1/2)  
    return hyp
```

In [5]:

```
# Let's try running it here for 5 and 12  
result = get_h(5,12)  
print(result)
```

13.0

What if we wanted to return more than one thing?

In [6]:

```
def get_h(a,b):  
    h = (a**2 + b**2)**(1/2)  
    h_times_2 = h * 2.0  
    return h, h_times_2
```

In [7]:

```
a = 3  
b = 4  
result, result_times_2 = get_h(a,b)  
print(result, result_times_2)
```

5.0 10.0

Variable scope (or "all the ways Python will try to drive me crazy")

FIRST, LET'S LOOK AT THE RECOMMENDED APPROACH

In [11]:

```
def get_h(a,b):  
    h = (a**2 + b**2)**(1/2)  
    return h
```

In [13]:

```
# Define my parameters/variables  
my_a = 3  
my_b = 4  
  
#Call my function  
result = get_h(my_a,my_b)  
print(result)  
  
# Will the code below work? If I haven't defined "h" anywhere outside my function, it won't work!  
# If "h" is only defined inside the function, the rest of my code doesn't know about it.  
  
# print(h)
```

5.0

THOUGHT EXERCISE: WILL THIS CODE WORK?

In [14]:

```
a = 3
b = 4
def get_h():
    h = (a**2 + b**2)**(1/2)
    return h
result = get_h()
print(result)
```

5.0

DISCLAIMER: DON'T GET IN THE HABIT OF WRITING CODE LIKE THIS!!

In [15]:

```
a = 3
b = 4
def get_h():
    h = (a**2 + b**2)**(1/2)
    return h
result = get_h()
print(result)
```

5.0

MY CODE IS NO LONGER EASILY REUSABLE LATER ON!

In [16]:

```
# Define new parameters/variables
new_a = 5
new_b = 12

#Call my function
result = get_h(new_a, new_b)
print(result)
```

TypeError

Traceback (most recent call last)

Cell **In[16], line 6**

```
3 new_b = 12
5 #Call my function
----> 6 result = get_h(new_a, new_b)
      7 print(result)
```

TypeError: get_h() takes 0 positional arguments but 2 were given

Also, when you use default argument values in your function, you can further confuse yourself if you're not being careful!

You'll see more about setting default arguments in the next PCA.

In [17]:

```
a = 3
b = 4
def get_h(a=5, b=12):
    h = (a**2 + b**2)**(1/2)
    return h
result = get_h()
print(result)
```

13.0

Goals for today

- Practice reading and writing `if` statements and more complex options that use `elif` and `else`
- Revisit the savings problem to calculate saving for many different people (*using `if` statements*)
- Re-write your code *using a function*.

Looking forward - Day 5 PCA: Getting more comfort and practice with functions

- Spend more time thinking about and learning about how to write functions
- Spend more time practicing writing functions and using functions